

PROJET : MISE EN PLACE D'UN PIPELINE DE DONNÉES (ETL)

Plateforme ELT & Backtesting sur les matières premières

Des marchés et des news financières à un laboratoire de stratégies :
Extract · Load · Transform · Backtest · Dashboard

Python

BigQuery

dbt

NLP

Streamlit

M1 Data & IA — Ynov · 2025-2026

Soutenance du 07/07/2026 · Durée : 10 minutes

Un pipeline de données au service d'une question métier

« Les actualités financières améliorent-elles une stratégie de trading technique sur les matières premières ? »

Pourquoi ce sujet

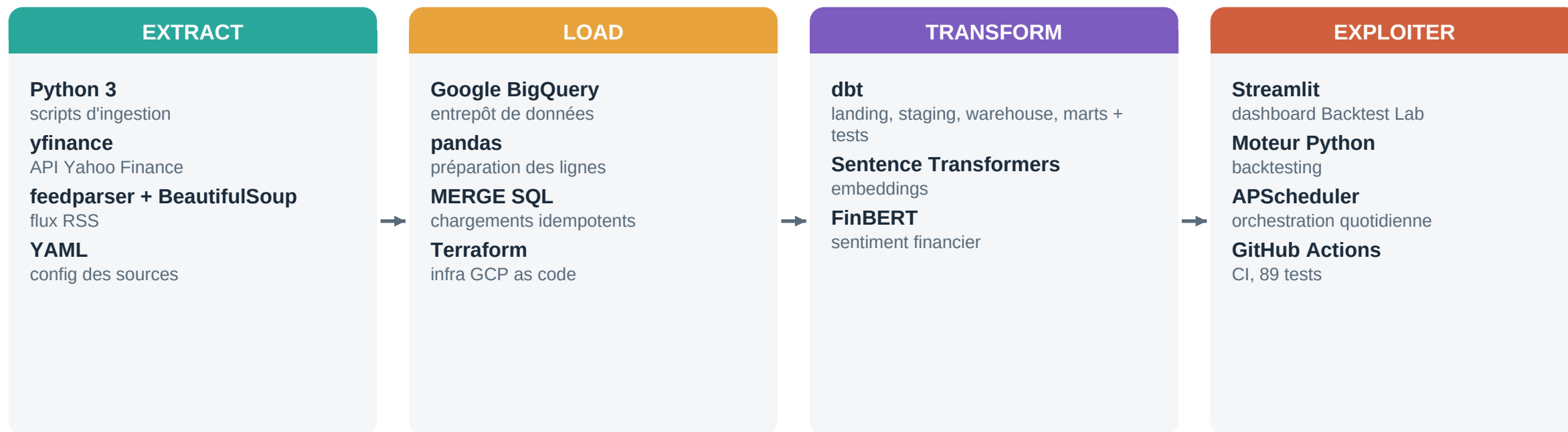
- **Marchés volatils** — or, pétrole, gaz, blé... sensibles aux chocs géopolitiques, d'offre et de météo.
- **Deux mondes de données** — prix de marché structurés (OHLCV) et actualités non structurées (RSS).
- **Volume justifiant un ETL** — 20 actifs, 5 ans d'historique, flux de news quotidiens.

Ce que nous avons construit

- **Un pipeline ELT complet** — extraction, chargement BigQuery, transformations dbt.
- **Un enrichissement NLP** — sentiment et pertinence des news par matière première.
- **Un moteur de backtesting** — 5 stratégies comparées à des benchmarks.
- **Un dashboard Streamlit** — pour simuler et comparer des portefeuilles.

Fil rouge : produire un jeu de données propre, reproductible et exploitable — puis le prouver avec le backtesting.

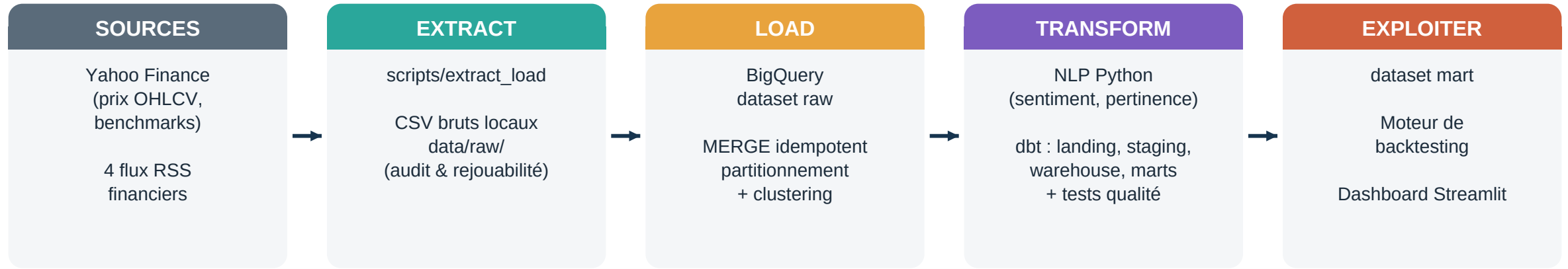
Stack technique : un outil par responsabilité



Choix clé : approche ELT plutôt qu'ETL classique

On charge d'abord le brut dans BigQuery, puis on transforme en SQL avec dbt : traçabilité totale et transformations rejouables.

Architecture globale : de la source au dashboard



Orchestration quotidienne (scripts/orchestrate.py)

ingestion → dbt staging → NLP → dbt warehouse/marts → dbt test → backtesting → logs

- **3 datasets BigQuery** — raw (brut), dbt_finance (intermédiaire), mart (tables finales).
- **Chaque étape est loguée** — run_id, statut, lignes traitées, erreurs — table pipeline_logs.

Trois familles de sources, un même contrat de sortie

MARCHÉS — Yahoo Finance

- 20 matières premières (or, pétrole, blé...) définies dans config/commodities.yml
- OHLCV quotidien depuis 2020 via yfinance
- Mode incrémental : reprise à MAX(date) + 1
- Fallback local si l'API échoue

BENCHMARKS — références

- Buy & Hold par actif (base 100)
- Indice synthétique équipondéré du panier (COMMODITY_INDEX)
- Proxy externe : ETF DBC
- Servent à juger les stratégies, pas à générer des signaux

NEWS — 4 flux RSS

- Investing.com, Barchart, CME Group, Nasdaq
- Parsing feedparser + nettoyage HTML (BeautifulSoup)
- URLs canoniques, article_id + content_hash
- Déduplication dès l'extraction

- **Sortie systématique** — un CSV horodaté dans data/raw/ (zone tampon pour audit, debug et rejeu) + chargement BigQuery.
- **Robustesse** — options --dry-run, --incremental, --skip-bigquery ; les tickers en échec partent dans un fichier d'erreurs dédié.

BigQuery raw : un chargement idempotent et peu coûteux

DATASET raw — 8 TABLES

market_data_raw

prix OHLCV des 20 actifs

benchmarks_raw

niveaux d'indices & poids

news_raw

articles RSS nettoyés

news_embeddings_raw

vecteurs des articles

news_sentiment_raw

scores FinBERT

article_commodity_relevance_raw

liens article-actif

news_features_raw

features news par jour

pipeline_logs_raw

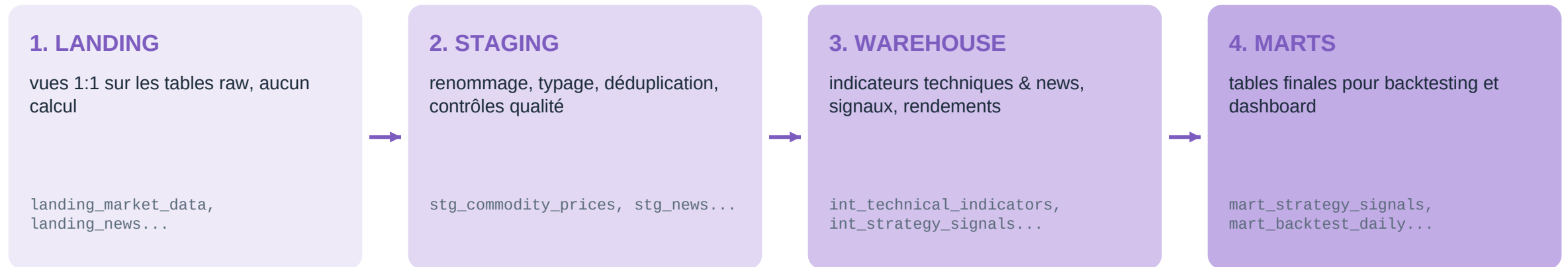
suivi des exécutions

Comment on charge

- **Create if not exists** — les scripts créent les tables avec le bon schéma au premier run.
- **MERGE plutôt qu'append** — clé symbol + date pour les prix, article_id pour les news : relancer le pipeline ne crée jamais de doublons.
- **Partitionnement par date** — et clustering (symbol, source...) pour limiter les octets scannés et les coûts.
- **Intégrité vérifiée** — schémas typés à l'écriture, fichiers d'erreurs, logs par étape dans pipeline_logs_raw.

**Résultat : une zone brute fiable et rejouable,
socle unique de toutes les transformations en aval.**

dbt : 4 couches SQL, du brut aux tables métier



- **Nettoyage exigé par le sujet** — doublons, valeurs manquantes, formats et noms de colonnes uniformisés — fait en SQL, versionné, testé.
- **Tests dbt critiques** — cohérence OHLC, pas de dates futures, fraîcheur des données, signaux valides, articles uniques.
- **Anti look-ahead bias** — `int_daily_returns` décale chaque signal d'un jour (lag) : un signal calculé à J s'exécute à J+1.

Enrichissement : le NLP en Python, l'agrégation en dbt

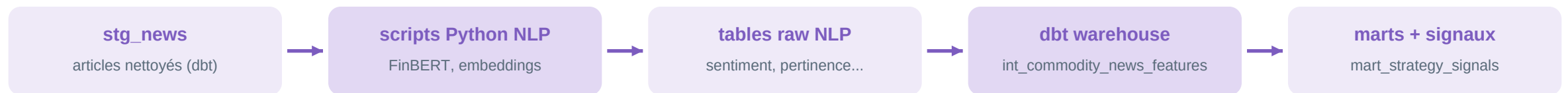
Pipeline NLP (Python)

- **Embeddings** — all-MiniLM-L6-v2 vectorise chaque article.
- **Pertinence** — similarité cosinus article ↔ matière première, seuil 0.35, plusieurs actifs possibles par article.
- **Sentiment** — FinBERT → score = P(positif) - P(négatif).
- **Features par actif et par jour** — news_pressure_score, surprise 20 j, risques géopolitique / offre / météo.

Indicateurs & signaux (dbt)

- **Indicateurs techniques** — SMA, RSI, Stochastic RSI, MACD, ATR, Bollinger, volatilité.
- **5 stratégies** — Buy & Hold, croisement SMA, SMA + Stoch RSI, breakout 20 j, technique + filtre news RSS.
- **Univers testé** — chaque matière première + l'indice COMMODITY_INDEX comme actif synthétique.

LE FLUX HYBRIDE :



Pourquoi ce découpage hybride ?

dbt excelle en SQL, pas pour charger FinBERT. Python écrit ses résultats dans raw, dbt reste seul maître des tables transformées — rejouable sans relancer le NLP.

Un pipeline complet, testé et reproductible

Ce que le projet livre

- Chaîne ELT de bout en bout : 3 familles de sources → BigQuery → dbt → marts
- Qualité outillée : 89 tests, tests dbt critiques, CI GitHub Actions, audit des secrets
- Méthode saine : exécution à J+1, frais & slippage, split calibration / validation / test
- Infra as code (Terraform) et orchestration quotidienne loguée

Limites assumées

- Futures continus Yahoo : le roll réel est simplifié
- Couverture RSS partielle, NLP perfectible
- Coûts et slippage volontairement simplifiés (MVP)

Place à la démo — Streamlit « Backtest Lab »

Simuler un portefeuille sur un actif ou sur COMMODITY_INDEX, croiser plusieurs stratégies, et vérifier face aux baselines : la stratégie bat-elle un simple Buy & Hold ?

Outils : Backtest · Comparaison